1. File kya hai, OS ke nazariye se

Sabse pehle ek mental model banao. Tumhara Go program ek **guest** hai, aur Operating System ek **hotel manager**. Disk pe padi files manager ki property hain — tum directly haath nahi laga sakte. Tumhe manager se request karni padti hai: *"Mujhe yeh file kholke do."*

Jab tum `os.Open("example.txt")` karte ho, andar ek **syscall** hota hai (Linux pe `open()`). OS check karta hai — file exist karti hai? Tumhare paas permission hai? Sab theek hai toh OS tumhe ek **file descriptor** deta hai — ek chhota sa integer, jaise hotel ka room key card. Go ka `*os.File` object basically isi file descriptor ke upar ek wrapper hai.

Isliye `os.Open` ka return signature hai:

```go
f, err := os.Open("example.txt")
```

`f` → `*os.File` ka **pointer** (key card) `err` → agar manager ne mana kar diya (file nahi mili, permission nahi hai)

Pointer kyun? Kyunki file ek shared resource hai — uski copy banane ka koi matlab nahi. Sab jagah usi ek file handle ko refer karna hai, isliye pointer.

---

2. Go ka error pattern — `if err != nil`

Tumne Java/JS me `try-catch` dekha hoga — wahan error **throw** hoti hai, upar se neeche girti hai jab tak koi catch nahi karta. Go ne yeh philosophy reject kar di. Go bolta hai: **error koi exception nahi hai, error ek normal return value hai.** Jaise function int return karta hai, waise hi error return karta hai.

```go
f, err := os.Open("example.txt")
if err != nil {
    panic(err)
}
```

`nil` ka matlab — "kuch galat nahi hua." Agar `err != nil` hai, matlab kuch toot gaya.

Analogy: Try-catch fire alarm jaisa hai — kahin bhi aag lagi, building bharki alarm bajega aur sab bhag denge. Go ka pattern hai ki har kaam ke baad worker tumhe ek receipt deta hai — "kaam ho gaya" ya "kaam fail hua, yeh reason hai." Tum har receipt khud check karte ho. Verbose hai, lekin **explicit** hai — code padhte hi pata chal jata hai ki kahan kya fail ho sakta hai.

`panic(err)` kya hai? Yeh Go ka "ab kuch nahi ho sakta, program crash karo" button hai. Production me tum panic *kabhi nahi* karoge file errors pe — tum log karoge, retry karoge, ya error upar return karoge (jaise tumhare notification service me hota hai). Video me sirf simplicity ke liye panic use hua hai.

---

3. `f.Stat()` — file ki kundali nikalna

```go
fileInfo, err := f.Stat()
```

`Stat` ek aur syscall hai (`stat()` Linux me). Yeh file ka **content nahi padhta** — sirf metadata padhta hai. Soch lo file ek courier parcel hai: `Stat` parcel kholta nahi, sirf label padhta hai — naam, weight, kab bheja gaya, kaun khol sakta hai.

`FileInfo` interface pe yeh methods milti hain:

```go
fileInfo.Name()    // "example.txt" — sirf naam, path nahi
fileInfo.Size()    // 12 — bytes me
fileInfo.IsDir()   // false — file hai ya folder
fileInfo.Mode()    // -rw-r--r-- — Unix permissions
fileInfo.ModTime() // last modified timestamp, timezone ke saath
```

Size 12 kyun aaya "Hello Golang" ke liye? Kyunki har character ek byte hai (ASCII range me). `H-e-l-l-o` = 5, space = 1, `G-o-l-a-n-g` = 6. Total 12 bytes. Yahan ek subtle baat — agar Hindi text hota ("नमस्ते"), toh har character 3 bytes leta (UTF-8 encoding), toh character count ≠ byte count hota. File size hamesha **bytes** me hoti hai, characters me nahi.

`Mode()` jo `-rw-r--r--` jaisa output deta hai — yeh Linux permission system hai: owner read+write kar sakta hai, group sirf read, others sirf read. Tumhare SMTP password encryption wale kaam me yeh relevant hai — sensitive files ki permissions tight rakhni padti hain.

4. Video ka pehla bug — relative path

Video me `go run files/files.go` chala folder ke *bahar* se, aur error aayi: `no such file or directory`. Yeh samajhna bahut important hai.

`"example.txt"` ek **relative path** hai. Relative kis cheez ke? **Current working directory** ke — yaani jis folder me terminal khada hai, file ke location ke nahi. Tumne script `files/` folder ke bahar se chalayi, toh Go ne `example.txt` ko *bahar wale* folder me dhoonda, jahan woh thi hi nahi.

Yeh production me bohot common bug hai. Tumhare Probus server pe agar binary `/opt/app/` se chalti hai aur code me `"templates/email.html"` likha hai, toh woh `/opt/app/templates/email.html` dhoondhega — chahe binary kahin bhi rakhi ho. Isliye production me absolute paths ya config-driven paths use karte hain.

---

5. File read karna — Buffer wala tareeka

Ab asli maza. File read karne ke liye humein content ko **memory me kahin rakhna** padega. Disk se data seedha tumhare variable me teleport nahi hota — usse ek container chahiye. Us container ko **buffer** kehte hain.

```go
buf := make([]byte, 10)
```

Yeh kya hai? Ek **byte slice** — 10 khali dabbon ki ek row, har dabba ek byte rakh sakta hai. `make` ne memory me 10 bytes allocate kar diye, sab zero se filled.

```go
n, err := f.Read(buf)
```

`f.Read(buf)` OS se bolta hai: *"file se data utha aur is container me bhar do, jitna fit ho."* Return values:

- `n` → kitne bytes actually padhe gaye
- `err` → koi problem? Specially `io.EOF` jab file khatam ho jaye

Ab video ka interesting moment: File 12 bytes ki thi, buffer 10 ka. Result? Sirf `Hello Golo` tak aaya — last ke 2 characters (`n` , `g`) cut gaye. Kyun? Kyunki **container 10 ka tha toh 10 hi bhara.** Read function ne koi error nahi di — usne apna kaam kiya, jitni jagah thi utna bhar diya.

Analogy: Tum tanker se paani la rahe ho aur tumhari balti 10 litre ki hai. Tanker me 12 litre hai. Ek trip me 10 hi aayega. Ya toh badi balti lo (`make([]byte, fileInfo.Size())`), ya **multiple trips** maro (yahi streaming hai, section 9 me).

Print karne pe `buf` raw bytes dikhata hai — `[72 101 108 108 111 ...]`. Yeh ASCII values hain: 72 = 'H', 101 = 'e'. Bytes ko readable banane ke liye `string(buf)` karna padta hai — yeh bytes ko UTF-8 text ki tarah interpret karta hai. Computer ke liye sab numbers hain; "text" sirf humara interpretation hai.

---

6. `os.ReadFile` — shortcut, aur uska hidden danger

```go
data, err := os.ReadFile("example.txt")
fmt.Println(string(data))
```

Bas. Ek line. Na open, na close, na buffer banana — sab internally ho gaya. Toh phir upar wala lamba tareeka kyun sikhaya?

Kyunki `ReadFile` poori file ek saath RAM me load karta hai. 12 bytes ki file? Zero problem. 5 GB ki video file ya tumhare bulk email campaign ka 2 GB CSV export? **Server ki RAM khatam, application crash, OOM kill.**

Analogy: Tumhe ek kuan se paani tank me bharna hai. `ReadFile` bolta hai — *poora kuan ek hi bucket me utha lo.* Chhota gadda hai toh chalega; kuan hai toh bucket phategi. Streaming bolti hai — chhoti balti se 100 trips maro. Slow dikhta hai, lekin kabhi fail nahi hoga.

Rule of thumb: Config files, templates, chhoti JSON — `ReadFile`. Logs, exports, uploads, unknown-size user files — streaming.

7. Folder read karna — `ReadDir`

```go
dir, err := os.Open(".")
defer dir.Close()

fileInfos, err := dir.ReadDir(-1)
for _, fi := range fileInfos {
    fmt.Println(fi.Name(), fi.IsDir())
}
```

Yahan ek twist hai — `os.Open` se sirf files nahi, **folders bhi khulte hain**. Linux philosophy: *everything is a file*. Folder bhi ek special file hai jiska content "entries ki list" hai.

`ReadDir(n)` ka `n` parameter:

- `n > 0` → maximum `n` entries do ("pehle 2 dikhao")
- `n <= 0` → **saari** entries do

Video me `"."` matlab current folder, `"../"` matlab ek level upar (parent). Har entry pe `IsDir()` se check kar sakte ho — folder hai ya file. Tumhare use case me yeh tab kaam aayega jab, maan lo, ek `attachments/` folder ke saare files iterate karke email me attach karne hain.

8. File create aur write karna

```go
f, err := os.Create("example2.txt")
if err != nil { panic(err) }
defer f.Close()

f.WriteString("Hi Go")
```

`os.Create` ka behavior dhyan se samjho — yeh teen cheezein karta hai:

1. File **nahi hai** → bana deta hai
2. File **already hai** → uska content **truncate** (zero) kar deta hai — purana data udd jata hai!
3. Tumhe writable file handle de deta hai

Video me ek confusion tha jab same program run me do baar `WriteString` call hua aur dono strings file me aa gayi — "Hi Go" aur "Nice language". Yeh "append mode" nahi hai jaise video me bola gaya. Yeh isliye hua kyunki **file handle ek cursor maintain karta hai.** Pehli write ke baad cursor position 5 pe tha, doosri write wahan se continue hui. Same open session me consecutive writes natural hai. Agar program *dobara* run karte, toh `os.Create` file truncate kar deta aur fresh start hota. Real append mode (program restarts ke across) ke liye `os.OpenFile` with `os.O_APPEND` flag chahiye — yahi video ka homework task tha.

Bytes se write karna:

```go
bytes := []byte("Hello Golang")  // string → byte slice conversion
f.Write(bytes)
```

`WriteString` internally yahi karta hai. File me hamesha bytes hi jaate hain — string toh humari convenience hai.

9. `defer` — sabse important concept is poore lesson ka

```go
f, err := os.Open("example.txt")
if err != nil { panic(err) }
defer f.Close()
```

`defer` bolta hai: *"yeh function abhi mat chalao — jab current function (yahan `main`) exit ho raha ho, tab chalana. Chahe normally exit ho ya panic se."*

Close kyun zaroori hai? Yaad karo hotel manager wali analogy — file descriptor ek key card hai. OS ke paas **limited key cards** hain (Linux pe default per-process limit ~1024). Agar tum files kholte raho aur band na karo, toh ek point pe OS bolega: *"too many open files"* — aur tumhari poori service nayi connections/files accept karna band kar degi. **Yeh production outage ka classic reason hai.** Tumhare bulk email worker pool me agar har email ke liye template file khulti hai aur close nahi hoti, toh 1024 emails ke baad pipeline mar jayegi.

`defer` open ke turant baad kyun likhte hain, last me kyun nahi? Video ne yeh sahi bola — agar beech me koi `panic` ya early `return` aa gaya, toh last wali manual `Close()` line tak execution pahunchega hi nahi. Lekin `defer` register ho chuka hai toh woh **guaranteed** chalega. Pattern yaad rakho: **open → error check → defer close. Teeno ek saath, hamesha.**

(Ek subtle order bhi hai: `err` check *pehle*, `defer` *baad me* — kyunki agar open hi fail hua toh `f` nil hai, nil pe `Close()` defer karna galat hai.)

---

10. Streaming copy — bufio ke saath (lesson ka climax)

Ab woh part jo tumhare production pipeline se sabse zyada resonate karega. Goal: ek file ka data doosri file me copy karna, **bina poori file memory me load kiye.**

```go
sourceFile, err := os.Open("example.txt")
if err != nil { panic(err) }
defer sourceFile.Close()

destFile, err := os.Create("example2.txt")
if err != nil { panic(err) }
defer destFile.Close()

reader := bufio.NewReader(sourceFile)
writer := bufio.NewWriter(destFile)

for {
    b, err := reader.ReadByte()
    if err != nil {
        if err.Error() != "EOF" {
            panic(err)
        }
        break
    }
    e := writer.WriteByte(b)
    if e != nil { panic(e) }
}

writer.Flush()
```

Ab isko layer by layer kholte hain.

`bufio` kya problem solve karta hai?

Direct `f.Read()` har call pe **syscall** karta hai — yaani har baar OS manager ke paas jaana. Syscall mehenga hai (context switch, kernel mode). Agar tum byte-by-byte directly file se padho, toh 1 MB file = 10 lakh syscalls. Bekaar slow.

`bufio.NewReader` beech me ek **smart middleman** rakh deta hai jiske paas **4096 bytes (4 KB) ka internal buffer** hai (video me dikhaya tha — `defaultBufSize = 4096`). Ab kya hota hai:

- Tum `ReadByte()` bolte ho → bufio ek hi syscall me **4096 bytes ek saath** utha leta hai
- Agle 4095 `ReadByte()` calls **memory se** serve hote hain — zero syscall, almost free

- Buffer khali hua → next 4 KB chunk uthao

Analogy: Tumhe roz 1 chamach cheeni chahiye. Direct read = roz kirane ki dukaan jaana ek chamach ke liye. Buffered read = ek baar 1 kg ka packet le aao, ghar ke dabbe se chamach bharo. Dukaan trips 1000x kam.

`NewWriter` ulta same kaam karta hai — tum `WriteByte` karte raho, woh apne 4 KB buffer me jama karta rehta hai, buffer bharne pe ek hi syscall me disk pe phenk deta hai.

EOF — file khatam hone ka signal

`ReadByte` jab file ke end pe pahunchta hai toh ek **special error** return karta hai: `io.EOF` (End Of File). **Yeh failure nahi hai** — yeh ek polite signal hai: *"data khatam, ab kuch nahi bacha."* Isliye code me distinction hai:

- Error hai **aur EOF nahi hai** → kuch genuinely toot gaya → panic
- Error hai **aur EOF hai** → normal ending → `break` karke loop se bahar

Agar yeh check nahi karte aur sirf `err != nil` pe panic karte, toh **har successful copy ke end pe program crash hota** — kyunki EOF toh aayega hi aayega. Aur agar break nahi karte toh infinite loop. (Production note: `err.Error() != "EOF"` string comparison fragile hai — idiomatic Go me `errors.Is(err, io.EOF)` ya seedha `err == io.EOF` likhte hain.)

`Flush()` — bhoolne wali sabse khatarnak line

Yaad karo, writer apne buffer me data **jama** karta hai aur tabhi disk pe likhta hai jab buffer bhar jaye. Ab socho file 12 bytes ki hai aur buffer 4096 ka — buffer **kabhi bhara hi nahi!** Saara data writer ke memory buffer me baitha hai, disk pe **kuch nahi gaya.**

`writer.Flush()` bolta hai: *"buffer me jo bhi bacha hai, abhi turant disk pe likho, chahe buffer full ho ya nahi."*

Flush bhool gaye toh: program successfully chalega, koi error nahi, lekin destination file **khali ya incomplete** milegi. Yeh debugging me ghanton khane wala bug hai kyunki kahin koi error nahi dikhti. Analogy: tumne courier ke collection box me letters daal diye, lekin pickup truck aaya hi nahi — letters box me hi reh gaye, kabhi deliver nahi hue.

Variable shadowing wala chhota bug

Video me loop ke andar inner write error ko `err` naam dene pe compiler conflict hua, isliye `e` rakha. Yeh Go ka scoping issue tha — outer `err` already use ho raha tha loop condition logic me. Chhoti baat hai, par real coding me roz milti hai.

11. `io.Copy` — jo video ne sirf mention kiya

Video ke end me ek hint tha: agar sirf copy karna hai, toh yeh poora loop likhne ki zaroorat nahi:

```go
io.Copy(destFile, sourceFile)
```

Yeh internally wahi chunked streaming karta hai (32 KB chunks me), optimized aur battle-tested. Manual loop sikhaya gaya taaki tum samjho **andar kya hota hai** — lekin real code me `io.Copy` use karoge. Yeh tumhari Dung Beetle / Zerodha reading se connect hota hai — large report files ko stream karna bina memory blow kiye, exactly yahi pattern hai.

---

12. File delete — `os.Remove`

```go
err := os.Remove("example2.txt")
if err != nil { panic(err) }
fmt.Println("File deleted successfully")
```

Ek baat notice karo — `Remove` **file handle pe method nahi hai**, `os` package ka function hai jo **path** leta hai. Delete karne ke liye file kholne ki zaroorat hi nahi. Logical hai — tum parcel ko khol ke nahi phekte, seedha uthake phek dete ho. Aur hamesha ki tarah, Remove error return karta hai (file nahi mili, permission nahi hai), jo tum check karte ho.

Poore lesson ka ek-line-me nichod

Kaam	Tool	Kab
File kholna	`os.Open`	Read karna ho
Metadata	`f.Stat()`	Size/permissions/time chahiye, content nahi
Chhoti file read	`os.ReadFile`	Config, templates — KBs me
Badi file read	`bufio.NewReader` + loop	GBs, unknown size, uploads
File banana	`os.Create`	⚠️ Existing file truncate hoti hai
Likhna	`WriteString` / `Write`	Buffered ho toh **Flush bhoolna mat**
Copy	`io.Copy`	Production me yahi
Delete	`os.Remove`	Path do, handle nahi chahiye
Hamesha	`defer f.Close()`	Open ke turant baad — fd leak = outage

Aur teen golden rules jo har section me repeat hue: **(1)** har operation error return karta hai, har error check hoti hai; **(2)** open kiya toh defer close karo, usi saans me; **(3)** buffered writer use kiya toh Flush karo, warna data hawa me reh jayega.

Agar chaaho toh next step me main isko tumhare actual Probus context se map kar sakta hoon — jaise email attachment streaming ya SES export files handle karna isi pattern se kaise hoga. Batao kis direction me jaana hai.